

Which Model Actually Answered? Silent Model Substitution, Deterministic Validators, and Multi-Model Routing in an LLM Paper Agent

Andi Agung Dwi Arya B

Program Studi Magister Teknik Informatika, Fakultas Teknik

Universitas Hasanuddin

Makassar, Indonesia

D082251054 · [\[TODO: email\]](#)

Abstract—Research agents increasingly call commercial large language models (LLMs) through vendor SDKs and report the model name they requested as the provenance of their output. We show that this label can be wrong: the GitHub Copilot CLI accepts any model name and, when the account is not entitled to it, silently serves a different model—`claude-opus-4.8-fast` was requested and `claude-haiku-4.5` answered, and a fictitious model name “succeeded” as well. Only the provider’s usage event names the model that answered. We build this observation into *paper-agent*, an open-source agent for scientific papers that (i) records the served model of every call, (ii) routes work to models by role, and (iii) wraps every model output in deterministic validators. Its structure-safe translation masks LaTeX into placeholders and emphasis tags whose multiset and number-gluing are verified per unit. On an eight-section IEEE paper (146 units, 4,673 words, 262 placeholders) we compare a three-model pipeline (draft, post-edit by a different model, back-translation QA by a local model, per-unit selection) with two single-model configurations. The pipeline keeps all 146 units structurally intact and glossary compliance at 99.0%, but its mean cross-lingual cosine (0.826 direct, 0.958 back-translation) is *not* higher than that of a single model (0.837, 0.961) and it costs three times the wall time. Its value is error catching: a sentence left in English, a wrong technical term, and placeholders moved in front of numbers were found only because a second model re-read the text and because validators rejected malformed output. Three error classes—moved placeholders, a QA model that echoed its input, and dropped placeholders—were detected by no model, including the judge, but by the validators. We argue that served-model provenance and deterministic validation should be standard in research agents, and that multi-model review is best applied selectively.

Index Terms—large language models, model routing, provenance, machine translation, LaTeX, compound AI systems, LLM-as-a-judge, research agents

I. INTRODUCTION

Scientific software built around LLMs is now routinely a *compound* system: several models, retrieval components and rule-based checks cooperate, and the choice of model per step is a design variable rather than a constant [1]–[3]. Research agents that read, translate, review or summarise papers are a prominent case, and their results enter theses and publications together with a statement of the kind “the LLM was `model-X`”. That statement is part of the scientific record: it decides whether a result can be reproduced and whether two runs are comparable.

This paper starts from an incident in such a system. A neuro-symbolic agent for detecting synthesis-gap indicators in scientific literature [4] called a commercial model through the GitHub Copilot SDK [5] and labelled every result with the configured model, `claude-opus-4.8-fast`. When the account’s entitlement changed, the CLI kept accepting the name—and kept returning answers—while a different model, `claude-haiku-4.5`, was doing the work. Nothing in the response signalled the substitution; a fictitious model name was accepted just as quietly. The only reliable evidence was a usage event that names the model that answered. The label in the thesis was, for a period, wrong.

The incident raises a general question for research agents: *which model actually answered, and how should an agent know, record and exploit that?* We turn the question into three research questions.

- **RQ1** Does the model that answers a request equal the model that was requested, and what mechanism gives an agent a truthful provenance label?
- **RQ2** On a structure-sensitive task—translating a LaTeX paper while keeping citations, mathematics and cross-references intact—does a pipeline that combines different models (translator, post-editor, judge) outperform a single model, and at what cost?
- **RQ3** Which errors are caught by models (a second model re-reading the output) and which only by deterministic validators?

We answer them with *paper-agent*, an open-source agent for scientific papers whose every LLM call records the model that answered, routes work to models by role, and validates every model output deterministically. The contributions are:

- an empirical account of silent model substitution in a vendor SDK and a provenance mechanism (served-model logging, substitution warnings, truthful labels) that we recommend for research agents;
- a structure-safe translation method for LaTeX that masks commands, mathematics and references into placeholders and emphasis into tags, verifies their multiset and their attachment to numbers per unit, and reproduces the source

byte for byte when no translation is applied;

- a controlled comparison of a three-model pipeline with two single-model configurations on the same 146 units, with per-unit metrics (cross-lingual cosine, back-translation cosine, glossary compliance, structural integrity, time, served models) released with the code;
- an error analysis showing that the pipeline’s benefit lies in catching errors rather than in raising averages, and that three error classes were invisible to every model, including the judge, but caught by validators.

The agent also verifies references against bibliographic databases, gates citations, builds and checks the PDF, reviews a paper with two independent critics, and converts Word manuscripts to IEEE LaTeX; these functions are described as part of the system but evaluated only informally here. The Indonesian version of this paper was produced by the agent itself.

II. RELATED WORK

a) Compound systems, routing and ensembles: Frugal-GPT showed that cascading cheap and expensive models can match a strong model at a fraction of the cost [2]; RouteLLM learns a router from preference data [3]; LLM-Blender ranks and fuses the outputs of several models [6]; Mixture-of-Agents layers models that refine each other’s answers [7]. Chen et al. observe that more calls do not monotonically help and that the benefit depends on the task’s difficulty distribution [1]. Our pipeline is a small compound system with fixed roles; our contribution is not a better router but the finding that the combination’s value on our task shows up in error catching rather than in mean quality, and that deterministic validators are the component that catches the most.

b) Models judging models: LLM-as-a-judge agrees with humans at the level of inter-annotator agreement on chat tasks [8], and iterative self-feedback improves many generation tasks [9], [10]. Two results caution against a single model judging itself: LLMs cannot reliably self-correct reasoning without external signals [11], and LLM evaluators recognise and favour their own generations [12]. We therefore use a *different* model as post-editor, a third one for back-translation, and never let a model be the sole arbiter of structural validity.

c) Machine-translation quality estimation: Reference-based metrics such as COMET [13] need human references we do not have; expert MQM annotation [14] is the gold standard but costly. Round-trip translation has been revisited as a reference-free estimator [15], GPT-class models are competitive translators [16] and state-of-the-art translation evaluators [17]. We combine a cross-lingual sentence-embedding similarity [18], [19] with back-translation by a local model [20]; back-translation as a technique originates in data augmentation [21]. Terminology consistency, enforced in neural MT through constrained decoding [22], is handled here by a glossary in the prompt and measured deterministically afterwards.

d) Provenance and model documentation: Model cards [23] document what a model is; hallucination surveys [24] document what it may fabricate. Both presuppose that the

system knows which model it is using. Vendor documentation lists which models each subscription tier may use [25] and independent leaderboards rank them for research-style tasks [26], but neither tells a running agent what it was served. ReAct-style agents [27] that act on tool results would act on a wrong model label just as readily; we add the served model to the trace.

III. SYSTEM DESIGN

Fig. 1 shows the translation path of paper-agent; the other commands (reference verification, citation gate, build QA, two-critic review, DOCX conversion, natural-language task planning) share the same engines, roles and provenance log. The agent is a Python package with a command-line interface and an HTTP API; every LLM call goes through one of two engines, a local Ollama server or the GitHub Copilot SDK [5].

A. Served-Model Provenance

The Copilot SDK runs the vendor’s CLI as a subprocess and exposes a session API. We observed that a session created with any `model` string answers; when the account is not entitled to the model, another model is used and no error or field in the response indicates it. The session, however, emits an `assistant.usage` event whose `model` field names the model that generated the tokens. The engine subscribes to this event for every call, returns the text together with the served model, counts requested and served models separately, and prints a one-time warning per (requested, served) pair. All reports produced by the agent (translation reports, review reports, task-run summaries) carry the served models. The local engine records the model name returned by the server, which is always the requested one.

B. Model Roles

Work is assigned to *roles*, not to models: DRAFT (translator), REVIEW (post-editor), QA (back-translator), FALLBACK, CRITIC and CRITIC2 (independent reviewers), PLANNER. The default binding reflects the models an entry-level Copilot subscription is entitled to—`gpt-5-mini` for drafting, `claude-haiku-4.5` for post-editing and critique—plus a local `gemma3:27b` [20] for QA, fallback and the second critic; the binding is overridden by an environment variable. Two rules are fixed: the post-editor is never the drafting model, and the two critics are never the same model [11], [12].

C. Structure-Safe Masking

Each line of a LaTeX section file is split into translatable *units*: a paragraph, the argument of `\section/\caption/\paragraph`, a table cell (the row is split on top-level `&`), or an `\item` body. Lines that are pure structure (`\begin`, `\label`, `\includegraphics`, rules, comments) are kept verbatim. Inside a unit, mathematics ($\dots$), citations, references, labels, identifiers (`\texttt`, `\textsc`), braces groups, `~`, `\%` and every other command with its arguments are replaced by numbered placeholders `[n]`; adjacent masked spans merge into one placeholder.

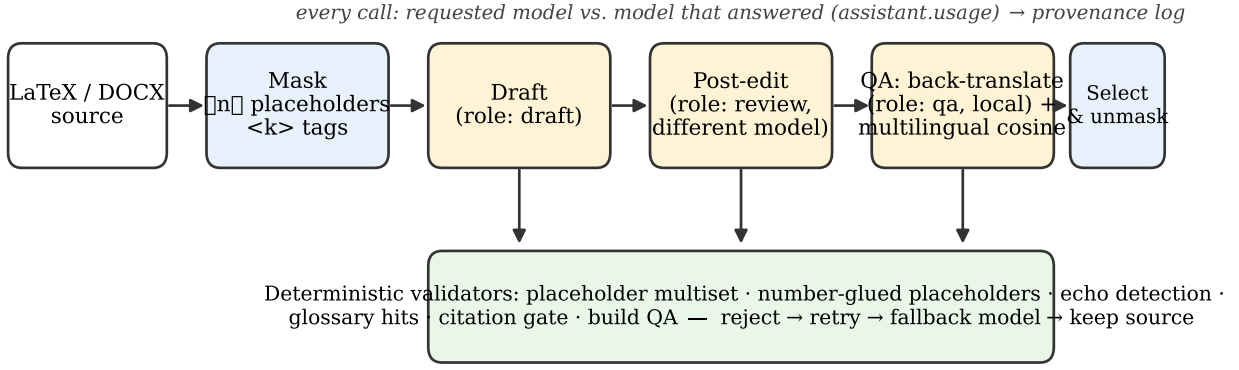


Fig. 1. Structure-safe translation in paper-agent. LaTeX is masked into placeholders and tags; three model roles (draft, post-edit, QA) are filled by different models; deterministic validators sit under every model call and reject malformed output (retry, fallback model, finally keep the source). Every call logs the requested and the served model.

The emphasis commands `\emph`, `\textbf` and `\textit` become inline tags `<k>...</k>` whose content is still translated. Unmasking reverses both, escapes bare specials the model may have typed (`%`, `&`, `_`), normalises typographic quotes and dashes back to LaTeX, and restores the unit’s leading and trailing whitespace. Masking followed by unmasking without translation reproduces every source file byte for byte; this is a unit test and a dry-run check of the tool. A final deterministic pass rewrites the word before `\ref` (“Fig.”, “Table”, “Section”) into the target language, since models occasionally leave it untranslated.

D. Deterministic Validators

Every model output for a unit is accepted only if (i) the multiset of placeholders equals that of the source, (ii) the multiset of tags equals that of the source, and (iii) every placeholder that is glued to a number in the source (“58 [3]” standing for 58%) is glued to the same number in the output (a decimal comma is tolerated). Rejected output is retried, then sent to the fallback model, and finally the source text is kept and the unit is reported as untranslated. A back-translation that equals its input is rejected as a failed QA call. After the run, glossary compliance is measured as the share of glossary source terms present in a unit whose target term appears in the output. The same spirit governs the other commands: a citation gate rejects any `\cite` key that is absent from the bibliography or not verified in at least two bibliographic sources, and the build step reports pages, undefined references and remaining TODO markers.

E. Pipelines and Selection

The MULTI pipeline runs DRAFT, then REVIEW with the source and the draft (the post-editor returns an improved text and a one-line note of what it changed), then QA: both candidates are back-translated into English by the local model and scored by $q = \frac{1}{2}(\cos(s, t) + \cos(s, b))$, where s is the masked source, t the candidate and b its back-translation, using the multilingual sentence-embedding model

paraphrase-multilingual-MiniLM-L12-v2 [19] with placeholders and tags stripped. The reviewed version is selected unless its score is more than 0.02 below the draft’s. A SINGLE pipeline uses one model for drafting and no post-editing; QA is still computed so that configurations are comparable. Units are sent in batches of about 2,500 characters as JSON; two batches run concurrently.

F. Other Commands

verify-refs matches each entry of a curated seed (title, authors, year, optional DOI or arXiv id) against Crossref, OpenAlex, Semantic Scholar, Scopus and arXiv; an entry is VERIFIED when title similarity ≥ 0.90 and year ± 1 agree in at least two sources, a wrong seed DOI is reported and replaced, and BibTeX is fetched from doi.org and validated against the seed. *review* sends the flattened paper to two critics with a fixed rubric and merges their findings, marking issues raised by both; each finding carries its serving model. *docx2tex* converts a Word manuscript to an IEEEtran project (headings, runs, tables, images, numeric citations, reference list as an unverified seed). *run* turns a natural-language request into a plan over these tools, produced by the planner model or by a keyword router, with all paths confined to the working directory.

IV. EXPERIMENTAL SETUP

a) Experiment A: substitution (RQ1): Through the Copilot SDK, with a fine-grained personal access token of an entry-level account, we requested models the account is entitled to (claude-haiku-4.5, gpt-5-mini), a model it is not entitled to (claude-opus-4.8-fast) and a fictitious name, and read the served model from the usage event (Section III-A). The behaviour is pinned by an automated test in the agent that originally hosted the incident [4].

b) Experiment B: translation configurations (RQ2): The material is the companion IEEE conference paper [4]: eight section files, 146 units, 4,673 words, 262 placeholders and 38 emphasis tags after masking; 51 of the units are table cells, 44 are headings or captions, 7 are list items and 44 are

TABLE I
CONFIGURATIONS COMPARED IN EXPERIMENT B. QA
(BACK-TRANSLATION + COSINE) IS COMPUTED FOR ALL THREE.

Configuration	Draft	Post-edit	Selection
MULTI	GPT-5 mini (Copilot)	Claude Haiku 4.5 (Copilot)	per unit by QA score
SINGLE-Haiku	Claude Haiku 4.5 (Copilot)	–	–
SINGLE-gemma3	Gemma 3 27B (local, 4-bit)	–	–

paragraphs. The target is formal Indonesian with a 103-term glossary derived from the thesis the paper summarises (e.g. *synthesis gap*, *rule engine* and *knowledge graph* are kept in English; *fragmentation* → *fragmentasi*). Three configurations translated the same units with the same glossary and prompts (Table I). Each configuration was run once; the API models run with the vendor’s default sampling, the local model at temperature 0.2. Hardware: two NVIDIA L40S (46 GB) for Ollama; gemma3:27b in 4-bit quantisation.

c) *Metrics*: (i) Structural integrity: units whose output passed the validators (Section III-D) and units left in English; (ii) glossary compliance; (iii) mean cross-lingual cosine between source and final text and between source and the back-translation of the final text; (iv) for MULTI, the post-editor’s edit ratio (1 – difflib similarity between draft and reviewed text) and the selection outcome; (v) wall time, model-seconds per unit, number of API calls and served models; (vi) agreement between configurations as mean character-level similarity of their final texts. Cosine values are computed by the same embedding model for all configurations and are reported with their standard error.

d) *Experiment C: error analysis (RQ3)*: We inspected every unit whose validators fired, every unit whose QA score dropped by more than 0.10 after post-editing, the post-editor’s change notes, and the lowest-scoring units of each configuration, and classified what caught each error: the post-editing model, the QA model, or a validator.

V. RESULTS

A. Experiment A: The Served Model Is Not the Requested Model

Table II summarises the substitution test. Entitled models were served as requested. Every non-entitled name—including a fictitious one—was accepted without error and answered by `claude-haiku-4.5`. No field of the response object differs between the two cases; only the usage event names the serving model. The agent that hosted the incident had labelled several months of analyses with the requested name; its labels are now derived from the usage event, a substitution warning is logged once per pair, and its health endpoint reports authentication explicitly, because an expired login had previously surfaced only as empty results.

TABLE II
EXPERIMENT A: REQUESTED VERSUS SERVED MODEL THROUGH THE
COPILOT SDK ON AN ENTRY-LEVEL ACCOUNT (ENTITLED:
CLAUDE-HAIKU-4.5, GPT-5-MINI, GPT-5.3-CODEX, GPT-4.1). NO
REQUEST RAISED AN ERROR.

Requested	Served (assistant.usage)
claude-haiku-4.5	claude-haiku-4.5
gpt-5-mini	gpt-5-mini
claude-opus-4.8-fast	claude-haiku-4.5
fictitious name	claude-haiku-4.5

B. Experiment B: Three Configurations, Same Units

Table III and Fig. 2 report the comparison. All three configurations produced a compilable Indonesian paper (nine pages against eight in English; zero undefined references or citations; the citation gate passed with the same 32 keys). On the automatic quality metrics the configurations are within one standard error of each other: the single Haiku 4.5 run has the highest mean cosine (0.837 direct, 0.961 back-translation) and glossary compliance (99.7%), the three-model pipeline follows (0.826, 0.958, 99.0%), and the local model is between them on cosine (0.831, 0.956) but lowest on glossary compliance (97.9%). The pipeline is the only configuration that never fell back to the fallback model, whereas the local model left two paragraphs in English after its placeholder output failed validation twice. Wall time differs by a factor of three (22 vs 7 vs 6.4 minutes) and API usage by a factor of two (32 vs 17 vs 0 calls). The final texts of any two configurations agree at 0.89–0.91 character similarity, i.e. the three systems produced largely interchangeable Indonesian.

a) *What the post-editor did*: In MULTI the post-editor changed 38 of 146 units (26%); the mean edit ratio over all units is 0.017, and over changed units 0.067 (maximum 0.625). The QA selection kept the reviewed text for 139 units and the draft for 7 (Fig. 3a); the two candidates lie close to the diagonal, i.e. post-editing rarely changed the automatic score. The change notes describe terminology and grammar corrections: *inferred* rendered as “diturunkan” (derived) was corrected to “disimpulkan”; *show* rendered as “dibuktikan” (proven) to “ditunjukkan”; a hyphenated calque “sembilan-aturan” to “dengan sembilan aturan”; and one complete English sentence that the drafting model had left untranslated in the opening paragraph was translated. None of these is visible in the mean cosine.

C. Experiment C: Who Caught Which Error

Table IV classifies the errors we found by the layer that caught them. Three classes were caught by validators and by nothing else. (i) *Moved placeholder*: in a pilot run the post-editor, while “tidying” number formats, moved the placeholder for % in front of the number (“hanya [[3]] 58” for “only 58%”); the multiset check passed, so the number-gluing rule was added and now rejects such output. (ii) *Echoed back-translation*: on 37 calls for short units (33 post-edited and 4 draft candidates) the local QA model returned the

TABLE III
EXPERIMENT B ON 146 UNITS (4,673 WORDS). COSINE VALUES ARE MEANS OF THE FINAL TEXT WITH STANDARD ERROR; INTEGRITY = UNITS THAT PASSED THE VALIDATORS WITHOUT FALLING BACK TO ENGLISH.

Metric	MULTI	SINGLE-Haiku	SINGLE-gemma3
Integrity (units)	146/146	146/146	144/146
Glossary compliance	284/287 (99.0%)	286/287 (99.7%)	281/287 (97.9%)
Cosine EN $\leftrightarrow$ ID	0.826 $\pm$ 0.011	0.837 $\pm$ 0.011	0.831 $\pm$ 0.011
Cosine EN $\leftrightarrow$ back-translation	0.958 $\pm$ 0.006	0.961 $\pm$ 0.005	0.956 $\pm$ 0.007
Units with English left	1 (quoted claim)	0	2
Wall time (s)	1,334	425	384
Model-seconds per unit (draft + post-edit)	7.2 $\pm$ 5.0	3.0	2.8
Copilot calls (served model)	16 gpt-5-mini + 16 claude-haiku-4.5	17 claude-haiku-4.5	0

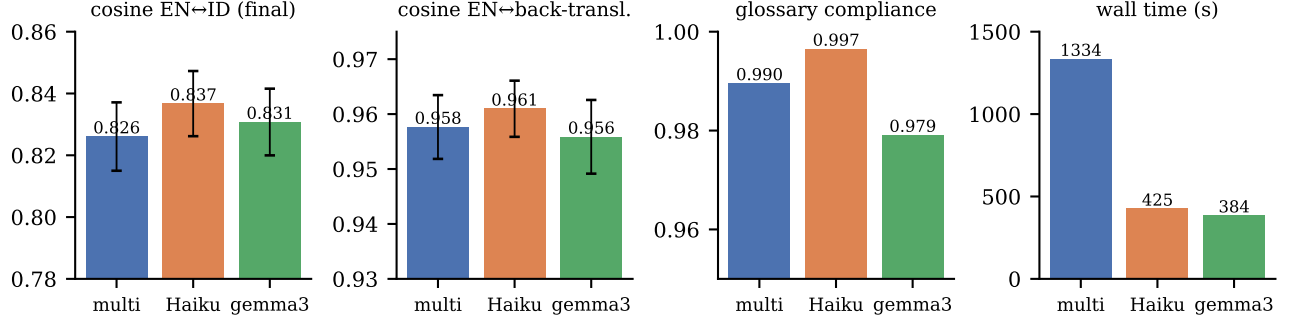


Fig. 2. Experiment B: final-text quality (cosine with standard error), glossary compliance and wall time of the three configurations.

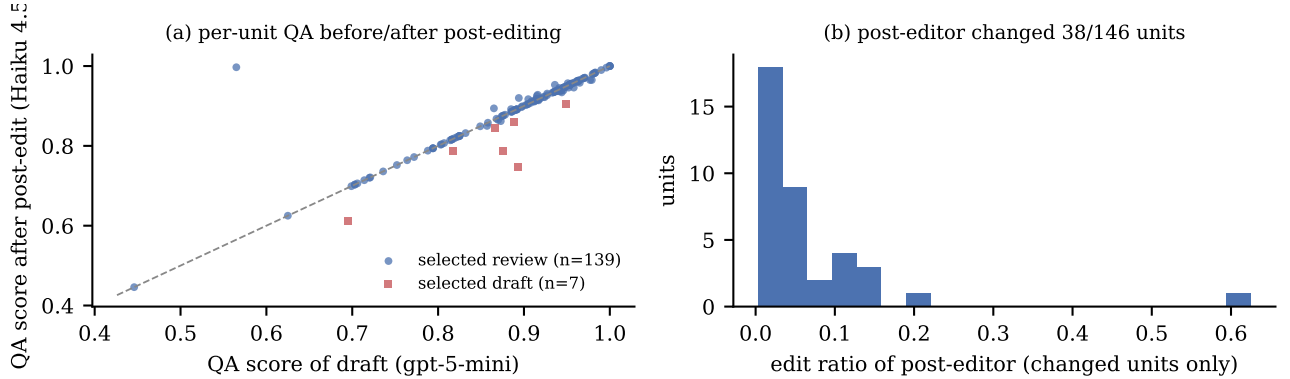


Fig. 3. MULTI pipeline: (a) per-unit QA score of the draft and of the post-edited text with the selected candidate; (b) distribution of the post-editor’s edit ratio over the 38 changed units.

Indonesian text unchanged instead of English; because the back-translation then equalled the candidate, the back-translation cosine collapsed (19 units dropped by more than 0.10) and the selector wrongly preferred the draft for 25 units whose two candidates were identical. The echo rule turns these into failed calls that are retried, after which the selection became 139/7. (iii) *Dropped placeholders*: the local model as sole translator lost placeholders in two paragraphs despite retries; without a fallback model these stay in English and are reported, which is the correct behaviour. Errors caught by a model were semantic: an untranslated sentence, wrong technical terms (*confounder* became “perusak” (destroyer) in the local run and “perancu” or “konfonder” elsewhere; “stokhastisitas” misspelled), a table cell

“Union” left in English and a heading truncated to “Kritikus lintas” in the single-Haiku run—the last three were found by human reading, which remains the final layer.

D. Answers to the Research Questions

RQ1. No: a vendor SDK can serve a different model from the requested one without any error, and the only truthful label is the model named in the usage event; the agent records it for every call. **RQ2.** The three-model pipeline did not raise mean quality—a single strong model matched or exceeded it at a third of the time—but it was the only configuration with full structural integrity and it corrected semantic errors that no metric showed. **RQ3.** Structural and protocol errors

TABLE IV

EXPERIMENT C: ERROR CLASSES AND THE LAYER THAT CAUGHT THEM (P = POST-EDITING MODEL, Q = QA MODEL, V = DETERMINISTIC VALIDATOR, H = HUMAN READING).

Error	Caught by	Occurrences
Placeholder moved before a number	V	1 unit (pilot)
QA model echoed its input	V	37 calls
Placeholders dropped (local model)	V	2 units
Sentence left untranslated	P	1 unit
Wrong technical term	P, H	4 terms
Misspelling (“stokhastisitas”)	H	1 unit
Cell or heading left/truncated	H	2 units (Haiku)

(placeholders, echoes) were caught only by validators; semantic errors were caught by a second model or by a human.

VI. DISCUSSION

a) Averages versus error catching: The three configurations are statistically indistinguishable on the automatic metrics, and the single strong model is the cheapest way to reach that level. Had we evaluated only by mean cosine we would have concluded that the multi-model pipeline is wasted effort. The per-unit view says otherwise: the pipeline is the configuration that never left a unit structurally broken, and its post-editor fixed semantic errors—an untranslated sentence, three wrong terms—that the metrics do not register because a sentence left in English still embeds close to its source. This matches the observation that the benefit of additional LLM calls depends on where the errors are, not on how many calls are made [1]: for a document in which most units are already translated well, a second model mostly agrees (108 of 146 units unchanged) and earns its cost on the few units it changes.

b) Validators are the cheapest layer and the only one that saw three error classes: Moved placeholders, echoed back-translations and dropped placeholders are not detectable by a model that reads fluent text, and the judge model was itself the source of one of them. A set-equality check, a regular expression for number-glued placeholders and a string comparison caught all three at negligible cost. This is the same lesson that motivated the companion system [4], where a symbolic rule layer validates LLM output: an LLM-as-a-judge [8] must itself be judged, and the judge of last resort should be deterministic where the property is formal. It also suggests a design for production use—*selective review*: run one strong model with validators on every unit, and send to a second model only the units that validators or cheap heuristics flag (low cosine, residual source-language function words, missed glossary terms, repeated placeholders). On our data such a filter would route roughly a tenth to a quarter of the units, approaching single-model cost with the pipeline’s error catching.

c) Local models for bulk, API models for the final pass: The local 27B model was the fastest, has no quota, and reached 97.9% glossary compliance, but it dropped placeholders in two

paragraphs and produced the only outright wrong technical term. In the pipeline it serves where its failure modes are cheap: back-translation for QA (where an echo is detected deterministically) and fallback. Which model fills which role should follow from the account’s entitlements and the task’s error profile, which is why roles, not model names, are the unit of configuration.

d) Provenance: Without reading the usage event, this paper would report Haiku’s work as Opus’s. The requested model is a configuration value; the served model is an observation, and only observations belong in a scientific record. We recommend that research agents (i) log the served model per call, (ii) label outputs with it, (iii) warn on substitution, and (iv) treat an authentication failure as a health failure rather than as an empty result. Model cards [23] describe models; the agent must still establish which card applies.

VII. LIMITATIONS

Each configuration was run once; LLM outputs vary between runs [4], and the differences we report between configurations (about 0.01 in cosine) are within that variability. All quality metrics are automatic; cross-lingual cosine and back-translation cosine are proxies that are known to miss terminology and fluency errors [14], [15], which is exactly where the post-editor acted—a human evaluation by bilingual domain experts is needed to quantify that benefit. One language pair, one document type and one glossary were studied. The QA model also serves as the fallback model, so a systematic weakness of `gemma3:27b` would affect two roles. Experiment A concerns one vendor SDK and one account tier at one point in time; the behaviour may change, and other providers may reject unknown models. The two-critic review and the DOCX converter are part of the released agent but were not evaluated here.

VIII. CONCLUSION

We documented that a vendor SDK can silently serve a different model from the one requested and turned the usage event that reveals it into a provenance mechanism for research agents. Around it we built paper-agent, which routes paper-handling tasks to models by role and validates every model output deterministically. In a controlled comparison on 146 LaTeX units, a three-model pipeline did not beat a single model on mean quality but was the only configuration with full structural integrity and corrected semantic errors invisible to the metrics; validators caught three error classes that no model, including the judge, detected. Future work is a human evaluation of the post-editor’s corrections, a learned filter for selective review, and extending the served-model provenance to other SDKs. The code, the per-unit metrics and both language versions of this paper are released at <https://github.com/devnolife/paper-agent>.

REFERENCES

- [1] L. Chen, J. Q. Davis, B. Hanin, P. D. Bailis, I. G. Stoica, M. Zaharia, and J. Zou, “Are More LLM Calls All You Need? Towards Scaling Laws of Compound Inference Systems,” 2024, arXiv:2403.02419.

- [2] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance,” 2023, arXiv:2305.05176.
- [3] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. G. Stoica, “RouteLLM: Learning to Route LLMs with Preference Data,” 2024, arXiv:2406.18665.
- [4] A. A. D. A. B., “A Neuro-Symbolic Agentic Approach for Detecting Synthesis-Gap Indicators in Scientific Literature,” Unpublished manuscript, Universitas Hasanuddin, 2026.
- [5] GitHub, “GitHub Copilot SDK for Python (github-copilot-sdk),” <https://github.com/github/copilot-sdk>, 2026, version 1.0.x, accessed Oct. 2026.
- [6] D. Jiang, X. Ren, and B. Y. Lin, “LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2023, pp. 14 165–14 178, doi: 10.18653/v1/2023.acl-long.792.
- [7] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, “Mixture-of-Agents Enhances Large Language Model Capabilities,” 2024, arXiv:2406.04692.
- [8] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, and Z. Wu, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” 2023, arXiv:2306.05685.
- [9] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, and L. Gao, “Self-Refine: Iterative Refinement with Self-Feedback,” 2023, arXiv:2303.17651.
- [10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: language agents with verbal reinforcement learning,” 2023, arXiv:2303.11366.
- [11] J. Huang, X. Chen, S. R. Mishra, H. Zheng, A. W. Yu, X. Song, and D. Zhou, “Large Language Models Cannot Self-Correct Reasoning Yet,” 2023, arXiv:2310.01798.
- [12] A. Panickssery, S. Bowman, and S. Feng, “LLM Evaluators Recognize and Favor Their Own Generations,” 2024, arXiv:2404.13076.
- [13] R. Rei, C. Stewart, A. C. Farinha, and A. Lavie, “COMET: A Neural Framework for MT Evaluation,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2020, pp. 2685–2702, doi: 10.18653/v1/2020.emnlp-main.213.
- [14] M. Freitag, G. Foster, D. Grangier, V. Ratnakar, Q. Tan, and W. Macherey, “Experts, Errors, and Context: A Large-Scale Study of Human Evaluation for Machine Translation,” *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 1460–1474, 2021, doi: 10.1162/tacl_a_00437.
- [15] J. Moon, H. Cho, and E. L. Park, “Revisiting Round-Trip Translation for Quality Estimation,” in *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*, 2020.
- [16] A. Hendy, M. Abdelrehim, A. Sharaf, V. Raunak, M. Gabr, H. Matsushita, Y. J. Kim, M. Afify, and H. H. Awadalla, “How Good Are GPT Models at Machine Translation? A Comprehensive Evaluation,” 2023, arXiv:2302.09210.
- [17] T. Kocmi and C. Federmann, “Large Language Models Are State-of-the-Art Evaluators of Translation Quality,” 2023, arXiv:2302.14520.
- [18] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 3980–3990, doi: 10.18653/v1/d19-1410.
- [19] —, “Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2020, pp. 4512–4525, doi: 10.18653/v1/2020.emnlp-main.365.
- [20] G. Team, A. Kamath, J. Ferret, S. Pathak, N. Vieillard, R. Merhej, S. Perrin, T. Matejovicova, A. Ramé, M. Rivière, L. Rouillard, T. Mesnard, G. Cideron, J.-b. Grill, S. Ramos, E. Yvinec, M. Casbon, E. Pot, I. Penchev, G. Liu, F. Visin, K. Kenealy, L. Beyer, X. Zhai, A. Tsitsulin, R. Busa-Fekete, A. Feng, N. Sachdeva, B. Coleman, Y. Gao, B. Mustafa, I. Barr, E. Parisotto, D. Tian, M. Eyal, C. Cherry, J.-T. Peter, D. Sinopalnikov, S. Bhupatiraju, R. Agarwal, M. Kazemi, D. Malkin, R. Kumar, D. Vilar, I. Brusilovsky, J. Luo, A. Steiner, A. Friesen, A. Sharma, A. Sharma, A. M. Gilady, A. Goedeckemeyer, A. Saade, A. Feng, A. Kolesnikov, A. Bendebury, A. Abdagic, A. Vadi, A. György, A. S. Pinto, A. Das, A. Bapna, A. Miech, A. Yang, A. Paterson, A. Shenoy, A. Chakrabarti, B. Piot, B. Wu, B. Shahriari, B. Petrini, C. Chen, C. L. Lan, C. A. Choquette-Choo, C. Carey, C. Brick, D. Deutsch, D. Eisenbud, D. Cattle, D. Cheng, D. Paparas, D. S. Sreepathihalli, D. Reid, D. Tran, D. Zelle, E. Noland, E. Huizenga, E. Kharitonov, F. Liu, G. Amirkhanyan, G. Cameron, H. Hashemi, H. Klimczak-Plucińska, H. Singh, H. Mehta, H. T. Lehari, H. Hazimeh, I. Ballantyne, I. Szpektor, I. Nardini, J. Pouget-Abadie, J. Chan, J. Stanton, J. Wieting, J. Lai, J. Orbay, J. Fernandez, J. Newlan, J.-y. Ji, J. Singh, K. Black, K. Yu, K. Hui, K. Vodrahalli, K. Greff, L. Qiu, M. Valentine, M. Coelho, M. Ritter, M. Hoffman, M. Watson, M. Chaturvedi, M. Moynihan, M. Ma, N. Babar, N. Noy, N. Byrd, N. Roy, N. Momchev, N. Chauhan, N. Sachdeva, O. Bunyan, P. Botarda, P. Caron, P. K. Rubenstein, P. Culliton, P. Schmid, P. G. Sessa, P. Xu, P. Stanczyk, P. Tafti, R. Shivanna, R. Wu, R. Pan, R. Rokni, R. Willoughby, R. Vallu, R. Mullins, S. Jerome, S. Smoot, S. Girgin, S. Iqbal, S. Reddy, S. Sheth, S. Pöder, S. Bhatnagar, S. R. Panyam, S. Eiger, S. Zhang, T. Liu, T. Yacovone, T. Liechty, U. Kalra, U. Evci, V. Misra, V. Roseberry, V. Feinberg, V. Kolesnikov, W. Han, W. Kwon, X. Chen, Y. Chow, Y. Zhu, Z. Wei, Z. Egyed, V. Cotruta, M. Giang, P. Kirk, A. Rao, K. Black, N. Babar, J. Lo, E. Moreira, L. G. Martins, O. Sanseviero, L. Gonzalez, Z. Gleicher, T. Warkentin, V. Mirrokni, E. Senter, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, Y. Matias, D. Sculley, S. Petrov, N. Fiedel, N. Shazeer, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, J.-B. Alayrac, R. Anil, Dmitry, Lepikhin, S. Borgeaud, O. Bachem, A. Joulin, A. Andreev, C. Hardin, R. Dadashi, and L. Hussenot, “Gemma 3 Technical Report,” 2025, arXiv:2503.19786.
- [21] R. Sennrich, B. Haddow, and A. Birch, “Improving Neural Machine Translation Models with Monolingual Data,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2016, pp. 86–96, doi: 10.18653/v1/p16-1009.
- [22] G. Dinu, P. Mathur, M. Federico, and Y. Al-Onaizan, “Training Neural Machine Translation to Apply Terminology Constraints,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019, pp. 3063–3068, doi: 10.18653/v1/p19-1294.
- [23] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, “Model Cards for Model Reporting,” in *Proceedings of the Conference on Fairness, Accountability, and Transparency*, ser. FAT* ’19. ACM, Jan. 2019, pp. 220–229, doi: 10.1145/3287560.3287596.
- [24] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, Mar. 2023, doi: 10.1145/3571730.
- [25] GitHub Docs, “Supported AI models in GitHub Copilot,” <https://docs.github.com/en/copilot/reference/ai-models/supported-models>, 2026, accessed 3 Oct. 2026.
- [26] Artificial Analysis, “Artificial Analysis: Independent LLM Evaluations (Intelligence Index, Terminal-Bench-Science, AA-Omniscience, AA-LCR),” <https://artificialanalysis.ai/evaluations>, 2026, accessed 3 Oct. 2026.
- [27] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” 2022, arXiv:2210.03629.